

CALCULS DE COMPLEXITÉ

```

1 def f1(n):
2     s = 0
3     for i in range(n):
4         s += i
5     return s

```

```

1 def f2(n):
2     c = 0
3     for i in range(n):
4         for j in range(n):
5             c += 1
6     return c

```

```

1 def f3(n):
2     c = 0
3     for i in range(n):
4         for j in range(i):
5             c += 1
6     return c

```

```

1 def f4(n):
2     i = 1
3     while i < n:
4         i *= 2

```

```

1 def f5(n):
2     c = 0
3     for i in range(n):
4         j = 1
5         while j < n:
6             c += 1
7             j *= 2
8     return c

```

```

1 def f6(n):
2     c = 0
3     for i in range(n):
4         if i % 2 == 0:
5             c += 1
6     return c

```

```

1 def f7(n):
2     s = 0
3     for i in range(n):
4         s += f2(i)
5     return s

```

```

1 def f8(n):
2     c = 0
3     for i in range(1, n):
4         for j in range(1, n // i):
5             c += 1
6     return c

```

```

1 def f9(n):
2     if n == 0:
3         return 1
4     return 2*f9(n-1)+1

```

```

1 def f10(n):
2     if n == 0:
3         return 1
4     return f10(n-1) + f10(n-1) + 1

```

Correction :

1. $f1$ est composée d'une boucle simple de n itérations, avec un coût constant à chaque itération, soit une complexité temporelle en $O(n)$.
2. La fonction est composée de deux boucles imbriquées indépendantes de n itérations chacune. La complexité est donc de l'ordre de n^2 .
3. La fonction est composée de deux boucles imbriquées dépendantes. Le nombre total d'itérations est $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$, avec un nombre constant d'opérations à chaque itération. On obtient une complexité temporelle en $O(n^2)$.
4. À chaque itération, i est multiplié par 2, jusqu'à ce que i dépasse n . Le nombre d'itération est le plus petit entier k tel que $2^k \geq n$, soit $k \approx \log_2(n)$. La complexité temporelle est donc en $O(\log(n))$.
5. La boucle *for* s'exécute n fois. La boucle *while* imbriquée s'exécute n fois (cf question précédente), soit une complexité de l'ordre de $n \log(n)$.
6. La fonction est composée d'une boucle *for* de n itérations. Chaque itération est composée d'une comparaison en temps constant et éventuellement d'une affectation de valeur à une variable. La complexité est donc linéaire en n .
7. La fonction $f7$ est composée de n itérations, avec un appel à $f2$ à chaque fois. $f2$ étant de complexité quadratique, la complexité de $f7$ est de l'ordre de n^3 .
8. La fonction est composée de deux boucles imbriquées et dépendantes. Le nombre total d'itération est $\sum_{i=1}^{n-1} \frac{n}{i} = n \sum_{i=1}^{n-1} \frac{1}{i}$. On reconnaît une somme harmonique, et on obtient une complexité asymptotique de l'ordre de $n \log(n)$.
9. $f9$ est une fonction récursive simple, avec un seul appel récursif, avec un nombre constant c d'opérations à chaque appel. La complexité est donc linéaire.

Calcul : en notant $T(n)$ le nombre d'opérations sur l'entrée n , T vérifie la relation de récurrence $T(n) = T(n-1) + c$, ce qui donne (suite arithmétique) une complexité linéaire en n .

10. $f10$ est une fonction récursive dans laquelle il y a deux appels récursifs à chaque appel. La relation de récurrence du nombre d'opérations de $f10(n)$ est $T(n) = 2T(n-1) + c$, ce qui donne une complexité exponentielle (de l'ordre de 2^n).